

PROBLEM STATEMENT

Existing code-review approaches typically rely on either narrow deterministic rule coverage or opaque, context-dependent AI-only findings and scoring. CodePilot combines deterministic and AI-assisted analysis in one platform to produce explainable, reproducible quality scores.

DATASET DESCRIPTION

Not a conventional supervised ML system — no training dataset is used. Evaluation relies on independently defined validation sets:

- **Source:** project-created controlled Python benchmark plus independently defined validation/holdout sets; ground truth fixed independently of CodePilot output.
- **Size:** 20-file primary benchmark (4 each: clean/low/medium/high/critical) · 12 model-property cases · 15-file unseen holdout (never used to tune the model).
- **Features:** LOC, cyclomatic complexity, severity, confidence, security/performance/semantic defects, maintainability characteristics.
- **Preprocessing:** ingestion → language/syntax validation → LOC & complexity extraction → finding normalization → confidence assignment → deterministic scoring.

METHODOLOGY / ARCHITECTURE

User → React Frontend → FastAPI Backend → Language Detection / Syntax Validation → Static + Semantic Analysis → Gemini AI Analysis → Hybrid Engine → Confidence Engine → CodePilot Scoring Engine → PostgreSQL / Redis → Results Dashboard

- **Static analysis:** deterministic rules for common defect patterns.
- **Semantic analysis:** deeper, language-specific rules across all six languages.
- **Gemini analysis:** complementary contextual reasoning (does not set the final score).
- **Hybrid Engine:** merges and deduplicates deterministic + AI findings.
- **Scoring Engine:** independently converts findings and code metrics into deterministic, bounded, explainable scores.

CodePilot Scoring Engine — Validation

custom deterministic scoring model, grounded in established software-quality concepts

FORMULA SUMMARY

Custom CodePilot deterministic formulas, grounded in established software-quality/measurement concepts (not ISO/CVSS/SQALE/Radon equations).

Impact_i = SeverityWeight_i × (Confidence_i ÷ 100)

Weights — Critical 10 · High 7 · Medium 4 · Low 1

NormalizedImpact = TotalImpact ÷ LOC_K

IssueDensity = Σ(Confidence_i ÷ 100) ÷ LOC_K

AvgComplexity = TotalCyclomatic ÷ #Functions

MaintainabilityScore = clamp(100 − [1.5·NormImpact + IssueDensity + AvgComplexity/2], 0, 100)

TechDebtHealth = clamp(100 − 2·NormImpact, 0, 100)

Security = clamp(100 − 3·SecImpact, 0, 100) · Performance = clamp(100 − 3·PerfImpact, 0, 100)

Overall = 0.30·Security + 0.25·Performance + 0.30·Maintainability + 0.15·TechDebtHealth

MATHEMATICAL VALIDATION

✓ Boundedness [0,100] PASS

✓ Severity monotonicity PASS

✓ Confidence monotonicity PASS

✓ Non-improvement when issues added PASS

✓ Complexity monotonicity PASS

✓ Exact weighted aggregation PASS

All weights sum to 1.0; the tested formulation prevents higher severity, higher confidence, additional findings, or higher complexity from improving the corresponding score.

TEST COVERAGE

▪ 46 scoring-engine tests passing (post performance-category fixes)

▪ 75 Python-analyzer tests passing (post N+1 detection fixes)

EXTERNAL EMPIRICAL VALIDATION

Dimension	Reference	n	ρ	p-value
Maintainability	Radon Maintainability Index	20	+0.3189	0.1707
Security	Independent benchmark severity	20	−0.3853	0.0935
Performance	Unseen holdout (15 files)	15	−0.7890	<0.001

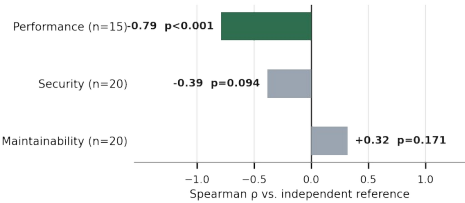
Maintainability: weak positive association, not significant. Security: negative association in the expected direction, not significant. Performance: strong negative association in the expected direction, significant on the unseen holdout. Strongest out-of-sample evidence is for Performance; Security and Maintainability remain preliminary.

Performance holdout: Precision 100% · Recall 62.5% (5 TP / 3 FN).

SENSITIVITY ANALYSIS

- 27 Maintainability coefficient configurations · 6 Overall-weight configurations tested
- **Baseline correlation ranges:** Maintainability ρ 0.9392–0.9989 · Overall ρ 0.9936–0.9967
- **Max score change:** 25.75 pts · **Max ranking reversals:** 39 / 190 pairs

Robust under tested parameter perturbations — this demonstrates parameter stability, not universal correctness of the coefficients.



Novelty: integration and implementation of deterministic multi-language semantic analysis, complementary Gemini contextual reasoning, Hybrid Engine fusion, confidence-aware processing, and an independent explainable deterministic scoring engine within one six-language code-review platform.

Conclusion: CodePilot is a multi-language hybrid code-review platform with a custom deterministic scoring model, grounded in established software-quality concepts, mathematically validated for boundedness and monotonicity, stress-tested for parameter stability, and empirically evaluated against independent references. The strongest out-of-sample evidence was observed for Performance (ρ = −0.789, p < 0.001), while Security and Maintainability evidence remains preliminary. The model is best characterized as a validated engineering metric for the project's scope.